

CodePipeline and CI/CD

Pipeline Stages · CodeBuild · Docker · ECR · Blue/Green · Smoke Tests

AWS Tutorial · ShopFlow

16-00

Chapter Overview — CI/CD at ShopFlow

ShopFlow CI/CD pipeline: GitHub (source) → CodePipeline (orchestration) → CodeBuild (build + test) → ECR (image registry) → CodeDeploy (blue/green ECS deploy). Pipeline runs on every merge to main. Target: code change to production in under 15 minutes with zero-downtime blue/green deployment.

Pipeline Stage	Service	Duration	Failure Action
Source	GitHub via CodeStar Connection	~5s (webhook trigger)	N/A
Build + Unit Tests	CodeBuild (build.yaml)	~4 min (Maven compile + test)	Block; notify Slack
Docker Build + ECR Push	CodeBuild (docker.yaml)	~3 min (multi-stage Docker build)	Block; notify Slack
Integration Tests	CodeBuild (integration.yaml)	~3 min (Spring Boot test with Testcontainers)	Block; notify Slack
Deploy to Staging	CodeDeploy ECS blue/green	~3 min (task replacement + health check)	Block; rollback
Smoke Tests (Staging)	CodeBuild (smoke.yaml)	~1 min (canary check via Synthetic Monitoring)	Block; rollback
Manual Approval (Production)	CodePipeline Manual Approval	Human decision	Block pipeline; notify approver
Deploy to Production	CodeDeploy ECS blue/green	~5 min (CANARY_10PERCENT_5MINUTES)	Block; notify Slack

ShopFlow CI/CD Pipeline

GitHub: main branch push → webhook

CodePipeline: Source Stage → CodeBuild → ECR → CodeDeploy

Build: mvn package -DskipTests (compile + package JAR)

Test: mvn test (unit tests + JaCoCo coverage gate 80%)

Docker: docker build -t shopflow/product-svc:SHA .

Push: ECR login + docker push (multi-arch: amd64 + arm64)

Deploy: CodeDeploy ECS blue/green (staging)

Smoke: curl /actuator/health + search canary

Manual Approval: Slack notification → ops approves

Deploy: CodeDeploy ECS production (CANARY_10PERCENT_5MINUTES)

16-01

CodePipeline Structure CDK

CodePipeline orchestrates the CI/CD workflow. Each pipeline has stages; each stage has actions. ShopFlow uses a standard pipeline (not CDK Pipelines) for the application pipeline to maintain fine-grained control over each stage and action configuration. CDK Pipelines is used for infrastructure (Ch15).

CodePipeline CDK

```
const pipeline = new codepipeline.Pipeline(this,"ShopFlowPipeline",{
  pipelineName: "shopflow-product-svc",
  restartExecutionOnUpdate: true, // re-run if pipeline itself is updated
  artifactBucket: artifactBucket, // S3 bucket for pipeline artifacts
  crossAccountKeys: false, // same account pipeline; no KMS needed
});
// Stage 1: Source – GitHub via CodeStar Connection
const sourceOutput = new codepipeline.Artifact("SourceOutput");
pipeline.addStage({
  stageName: "Source",
  actions: [new codepipeline_actions.CodeStarConnectionsSourceAction({
    actionName: "GitHub_Source",
    owner: "shopflow-org",
    repo: "shopflow-product-svc",
    branch: "main",
    connectionArn: githubConnection.attrConnectionArn,
    output: sourceOutput,
    triggerOnPush: true,
  })],
});
```

Action Type	Category	Provider	ShopFlow Stage
CodeStarConnectionsSourceAction	Source	GitHub (CodeStar)	Source stage
CodeBuildAction	Build	CodeBuild	Build, Test, Docker, Smoke stages
ManualApprovalAction	Approval	Manual	Before prod deploy
CodeDeployEcsDeployAction	Deploy	CodeDeploy ECS	Staging and Production deploy stages
LambdaInvokeAction	Invoke	Lambda	Post-deploy validation Lambda (optional)

16-02

CodeBuild — Build and Test Stage

CodeBuild runs in a managed Docker container. ShopFlow uses a custom CodeBuild image based on Amazon Linux 2 with Java 21, Maven, Docker, and AWS CLI pre-installed. Build cache: Maven local repository cached in S3 between builds (reduces build time from 8 min to 4 min).

CodeBuild Project CDK — Build Stage

```
const buildProject = new codebuild.Project(this,"BuildProject",{
  projectName: "shopflow-product-svc-build",
  source: codebuild.Source.codeStarConnection({...}),
  environment: {
    buildImage: codebuild.LinuxBuildImage.AMAZON_LINUX_2_5,
    computeType: codebuild.ComputeType.MEDIUM, // 4 vCPU, 7GB RAM
    privileged: true, // required for docker build inside CodeBuild
  },
  cache: codebuild.Cache.bucket(cacheBucket, {prefix: "maven-cache"}),
  buildSpec: codebuild.BuildSpec.fromObject({
    version: "0.2",
    phases: {
      install: {commands: ["java -version", "mvn --version"]},
      pre_build: {commands: ["echo Restoring Maven cache"]},
      build: {commands: [
        "mvn package -DskipTests -pl product-svc",
        "mvn test -pl product-svc --batch-mode",
      ]},
      post_build: {commands: ["mvn jacoco:report", "cat product-svc/target/jacoco.xml"]}
    },
    reports: {
```

```
junit: {
  files: ["product-svc/target/surefire-reports/**/*.xml"],
  fileFormat: "JUNITXML",
}
},
cache: {paths: ["/root/.m2/**/*.xml"]},
}),
});
```

16-03

Docker Build and ECR Push

ShopFlow builds multi-architecture Docker images (linux/amd64 + linux/arm64) to support both x86 and Graviton2 (ARM) ECS tasks. Graviton2 ECS tasks cost 20% less with same or better performance. Multi-arch images use Docker Buildx manifests — single image tag works on both architectures.

Buildspec — Docker Multi-Arch Build and ECR Push

```
# buildspec-docker.yaml
version: "0.2"
env:
  variables:
    IMAGE_REPO_NAME: shopflow-product-svc
    AWS_ACCOUNT_ID: !Sub "${AWS::AccountId}"
    IMAGE_TAG: $CODEBUILD_RESOLVED_SOURCE_VERSION # Git SHA
  phases:
    pre_build:
      commands:
        - aws ecr get-login-password --region us-east-1 | docker login
        --username AWS --password-stdin $AWS_ACCOUNT_ID.dkr.ecr.us-east-1.amazonaws.com
        - docker buildx create --name shopflow-builder --use
        - docker buildx inspect --bootstrap
    build:
      commands:
        - docker buildx build
        --platform linux/amd64,linux/arm64
        --tag $AWS_ACCOUNT_ID.dkr.ecr.us-east-1.amazonaws.com/$IMAGE_REPO_NAME:$IMAGE_TAG
        --tag $AWS_ACCOUNT_ID.dkr.ecr.us-east-1.amazonaws.com/$IMAGE_REPO_NAME:latest
        --push
        --cache-from type=registry,ref=$IMAGE_REPO_NAME:buildcache
        --cache-to type=registry,ref=$IMAGE_REPO_NAME:buildcache,mode=max
        -f product-svc/Dockerfile .
```

Dockerfile — Multi-Stage Build

```
# Stage 1: Build (Maven + JDK 21)
FROM amazoncorretto:21-alpine AS builder
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline -q # cache dependencies
COPY src/ ./src/
RUN mvn package -DskipTests -q
# Stage 2: Runtime (JRE 21 only — no Maven, no JDK)
FROM amazoncorretto:21-alpine-jre
WORKDIR /app
COPY --from=builder /app/target/product-svc.jar .
EXPOSE 8080
# Use exec form (not shell form) for proper SIGTERM handling:
ENTRYPOINT ["java", "-jar", "-XX:+UseG1GC", "-XX:MaxRAMPercentage=75.0", "product-svc.jar"]
```

16-04

ECR Repository and Image Lifecycle

Amazon ECR stores Docker images. ShopFlow uses one ECR repository per microservice. Image scanning on push (ECR enhanced scanning via Inspector) detects OS and package vulnerabilities. Lifecycle policy: keep last 10 tagged images + delete untagged images older than 1 day (cleanup build cache layers).

ECR CDK

```
const productRepo = new ecr.Repository(this, "ProductSvcRepo", {
  repositoryName: "shopflow/product-svc",
  imageScanOnPush: true, // ECR enhanced scanning via Inspector v2
  imageTagMutability: ecr.TagMutability.IMMUTABLE, // tags cannot be overwritten
  encryptionKey: repoKmsKey,
  removalPolicy: RemovalPolicy.RETAIN, // keep images if CDK stack deleted
  lifecycleRules: [
    {
      description: "Keep last 10 tagged releases",
      maxImageCount: 10,
      tagStatus: ecr.TagStatus.TAGGED,
      tagPrefixList: ["v", "sha-"],
    },
    {
      description: "Delete untagged images after 1 day (build cache cleanup)",
      maxImageAge: Duration.days(1),
      tagStatus: ecr.TagStatus.UNTAGGED,
    },
  ],
});
```

ECR Feature	ShopFlow Config	Benefit
Image Scanning	Enhanced scanning (Inspector)	OS + library CVE detection; block deploy if CRITICAL CVE found
Tag Immutability	IMMUTABLE	Git SHA tags cannot be overwritten — audit trail integrity
Lifecycle Policy	Keep 10 tagged; delete untagged daily	Prevents unbounded ECR storage cost (\$0.10/GB/month)
ECR Replication	Replicate to us-west-2 for DR deploy	ECS in DR region pulls from regional ECR (no cross-region pull latency)
Pull-through Cache	Docker Hub public images cached in ECR	Avoids Docker Hub rate limits in CodeBuild; faster pulls

16-05

CodeDeploy ECS Blue/Green

CodeDeploy manages ECS blue/green deployments. Traffic shifts from blue (current) to green (new) target group via ALB listener rule weight updates. ShopFlow uses CANARY_10PERCENT_5MINUTES: 10% traffic to green, wait 5 minutes for CloudWatch alarms, then shift 100%. Auto-rollback if any alarm fires during the window.

CodeDeploy ECS Deployment CDK

```
const deployGroup = new codedeploy.EcsDeploymentGroup(this, "ProdDG", {
  service: productEcsService,
  blueGreenDeploymentConfig: {
    blueTargetGroup: productTgBlue,
    greenTargetGroup: productTgGreen,
    listener: httpsListener,
    testListener: testListener, // port 8443: integration tests hit green
    terminationWaitTime: Duration.minutes(60), // keep blue for 1hr after 100% shift
    deploymentApprovalWaitTime: Duration.minutes(5), // wait before 100% shift
  },
  deploymentConfig: codedeploy.EcsDeploymentConfig.CANARY_10PERCENT_5MINUTES,
  autoRollback: {
    failedDeployment: true,
    deploymentInAlarm: true,
    stoppedDeployment: true,
  },
  alarms: [albLatencyAlarm, ecs5xxAlarm, ecsHealthAlarm],
});
```

```
});
```

Deployment Config	Traffic Shift	Wait Time	Rollback Trigger
AllAtOnce	0% → 100% immediately	None	Alarm fires after 100% shift (all users affected)
CANARY_10PERCENT_5MINUTES	0% → 10% → 5min wait → 100%	5 minutes at 10%	Alarm fires at 10% exposure (90% still on old version)
LINEAR_10PERCENT_EVERY_1MINUTE	10%+10%+... per minute	1 min per step	Stop and rollback at any step if alarm fires
Custom (ShopFlow could use)	1% → 24hr → 100%	24 hours at 1%	Maximum caution — 1% exposure for 24 hours

16-06

CodeBuild — Integration and Smoke Tests

Integration tests run against a real database using TestContainers (starts PostgreSQL and Redis in Docker inside CodeBuild). Smoke tests run after staging deploy: call /actuator/health, run a product search, add an item to cart. All tests must pass before production approval gate.

Integration Test Buildspec

```
# buildspec-integration.yaml
version: "0.2"
phases:
  pre_build:
    commands:
      - docker pull postgres:15-alpine # pre-pull for TestContainers
      - docker pull redis:7-alpine
  build:
    commands:
      - mvn verify -Pintegration -pl product-svc
      - Dspring.datasource.url=jdbc:tc:postgresql:15:///shopflow
      - Dspring.redis.host=localhost
```

Smoke Test Buildspec

```
# buildspec-smoke.yaml (runs after staging deploy)
version: "0.2"
env:
  variables:
    STAGING_URL: https://staging.shopflow.com
  phases:
    build:
      commands:
        - |
          HEALTH=$(curl -sf $STAGING_URL/actuator/health | jq -r .status)
          [ "$HEALTH" = "UP" ] || (echo "Health check failed: $HEALTH" && exit 1)
        - |
          RESULTS=$(curl -sf "$STAGING_URL/search?q=laptop" | jq .totalHits)
          [ "$RESULTS" -gt 0 ] || (echo "Search returned 0 results" && exit 1)
        - echo "Smoke tests passed"
```

■ Use TestContainers for integration tests in CodeBuild — it starts real database containers inside the build environment. `spring.datasource.url=jdbc:tc:postgresql:15:///shopflow` tells the Testcontainers JDBC driver to start a PostgreSQL 15 container automatically (no URL, no port — Testcontainers manages it). Requires `privileged: true` in CodeBuild compute environment (for Docker-in-Docker). Advantage: integration tests run without any external dependency — no shared staging database, no test data contamination.

16-07

CodePipeline Notifications and Slack Integration

CodePipeline publishes events to EventBridge when stages start, succeed, or fail. A Lambda function receives EventBridge events and posts formatted messages to Slack. Deployment start and success go to #deployments; failures go to #alerts with the pipeline URL and CloudWatch log link.

Pipeline Notification CDK

```
// EventBridge rule: pipeline execution state changes
const pipelineNotificationRule = pipeline.notifyOn("PipelineNotifications",{
  notificationRuleName: "shopflow-pipeline-notifications",
  target: new chatbot.SlackChannelConfiguration(this,"SlackChannel",{
    slackChannelConfigurationName: "shopflow-deployments",
    slackWorkspaceId: "T01XXXXXX",
    slackChannelId: "C02XXXXXX", // #shopflow-deployments channel
    notificationTopics: [notificationTopic],
  }),
  events: [
    codepipeline.PipelineNotificationEvents.PIPELINE_EXECUTION_STARTED,
    codepipeline.PipelineNotificationEvents.PIPELINE_EXECUTION_SUCCEEDED,
    codepipeline.PipelineNotificationEvents.PIPELINE_EXECUTION_FAILED,
    codepipeline.PipelineNotificationEvents.ACTION_EXECUTION_FAILED,
  ],
});
```

Event	Channel	Message Content	On-Call Page?
Pipeline started	#deployments	Service name, triggering commit SHA + author	No
Stage failed: Build	#deployments + #alerts	Stage name, error message, CodeBuild log link	No
Stage failed: Deploy (prod)	#deployments + #alerts + PagerDuty	Stage name, error message, AWS CloudWatch Dashboard link, rollback status	Yes (P2)
Pipeline succeeded	#deployments	Total duration, image SHA deployed, approver name	No
Manual approval needed	#deployments + approver DM	Staging smoke test results, diff summary, approver name	Yes (DM to approver)

16-08

ECR Image Vulnerability Scanning

ECR Enhanced Scanning uses Amazon Inspector to scan images for OS package vulnerabilities (CVE database) and programming language library vulnerabilities (Java JARs, Python packages). ShopFlow blocks deployment if CRITICAL or HIGH CVEs are found in the image. Scan results available within 5 minutes of push.

ECR Scan Gate in CodeBuild

```
# In buildspec-docker.yaml post_build phase:
post_build:
  commands:
  - |
    echo "Waiting for ECR scan results..."
    sleep 60 # Wait for Inspector scan to complete
  - |
    CRITICAL=$(aws ecr describe-image-scan-findings
    --repository-name shopflow/product-svc
    --image-id imageTag=$IMAGE_TAG
    --query "imageScanFindings.findingSeverityCounts.CRITICAL" --output text)
    HIGH=$(aws ecr describe-image-scan-findings
    --repository-name shopflow/product-svc
    --image-id imageTag=$IMAGE_TAG
    --query "imageScanFindings.findingSeverityCounts.HIGH" --output text)
    echo "CVEs: CRITICAL=$CRITICAL HIGH=$HIGH"
    if [ "$CRITICAL" != "None" ] && [ "$CRITICAL" -gt 0 ]; then
      echo "CRITICAL CVEs found – blocking deployment"
      exit 1
    fi
```

Vulnerability Severity	CodePipeline Action	ShopFlow CVE Examples	Remediation
CRITICAL	Block deployment immediately	Log4Shell (CVE-2021-44228), OpenSSL RC4	Update dependency version in pom.xml

Vulnerability Severity	CodePipeline Action	ShopFlow CVE Examples	Remediation
HIGH	Block deployment; Slack alert	Spring Framework RCE, Jackson deserialization	Update + rebuild + redeploy within 24h
MEDIUM	Warning in Slack; does not block	Various library vulnerabilities	Track in JIRA; fix in next sprint
LOW / INFORMATIONAL	Log only; no alert	Minor issues; low exploitability	Fix in quarterly dependency update

16-09

Manual Approval Gate and Audit Trail

CodePipeline Manual Approval action pauses the pipeline and sends an approval request to an SNS topic. ShopFlow sends this via Chatbot to Slack and also emails the release manager. The approver clicks "Approve" in the Slack message (via ChatOps button) or in the AWS Console. Approval/rejection is logged in CloudTrail.

Manual Approval CDK

```
const approvalAction = new codepipeline_actions.ManualApprovalAction({
  actionName: "Approve_Production_Deploy",
  notificationTopic: approvalTopic,
  additionalInformation: `Deploy ${imageTag} to production?
  Staging smoke tests: PASSED
  Staging deploy: SUCCESS
  ECR scan: 0 CRITICAL CVEs
  Approve link:
  https://console.aws.amazon.com/codesuite/codepipeline/pipelines/shopflow-product-svc/view`,
  externalEntityLink: stagingHealthUrl,
});
```

Approval Artifact	Where Logged	Contents	Retention
Who approved	CloudTrail (PutApprovalResult API call)	Approver IAM ARN, timestamp, comment, pipeline name	7 years (CloudTrail S3 archive)
What was approved	S3 pipeline artifact (imageDefinitionImage)	Image SHA, service name, staging test results	90 days (S3 lifecycle)
Approval time	CodePipeline execution history	Time from approval request to approval action	90 days
Rejection reason	CloudTrail + CodePipeline console	Required rejection comment (validation: min 10 chars)	7 years

■ Require a comment on Manual Approval rejection. Without a comment, engineers reject without documenting why, losing valuable release history. Set up a convention: rejection comment must include ticket number and reason (e.g., "SHOP-1234: performance regression in load test, p99 latency 3.2s vs 1.8s baseline"). This creates an audit trail for compliance and retrospectives. Make the Slack approval message include the Staging CloudWatch Dashboard link so approvers review metrics before clicking Approve.

16-10

Branch Strategy and Environment Mapping

ShopFlow git branching: feature/* → pull request → develop → staging deploy → main → prod pipeline. Feature branches trigger CodeBuild for unit + integration tests only (no deploy). Develop branch deploys to staging automatically. Main branch requires manual approval for production. Hotfix/* branches deploy directly to production after abbreviated testing.

Branch	Pipeline Trigger	Stages Executed	Environment
feature/*	CodeBuild only (PR check)	Build + Unit Test + Integration Test	None (no deploy)
develop	Full pipeline (no manual approval)	Build + Test + Docker + Deploy	Staging (auto-deploy on merge)
main	Full pipeline + manual approval	Build + Test + Docker + Deploy + Approve + Deploy	Staging → Production
hotfix/*	Abbreviated pipeline (no integration test)	Build + Unit Test + Docker + Deploy	Production (emergency fix)
release/v*	Full pipeline + extended smoke test	All stages + extended 30-min smoke test	Staging → Production

CodeBuild provides managed build environments with different compute sizes, operating systems, and runtimes. ShopFlow uses a custom ECR-hosted build image based on Amazon Linux 2 with Java 21, Maven 3.9, Docker 24, and AWS CLI 2 pre-installed. S3 caching for Maven `~/m2` reduces dependency download time from 4 min to under 1 min.

Cache Layer	What Is Cached	Cache Key	Time Saved
Maven dependencies (S3)	~/m2/repository — all JAR dependencies	Cache path: /root/.m2/**/*	3-4 min (download JARs on first)
Docker layer cache (ECR)	Docker image layers for base image	ECR buildcache tag	1-2 min (base image layers already)
Node.js modules (S3)	node_modules (CDK synth stage)	Cache path: **/node_modules/**	30s-1 min (npm ci uses cached)
No cache — always fresh	Test results, compiled classes	N/A — built fresh every time	N/A

CodeBuild Cache CDK

```
const buildProject = new codebuild.Project(this, "BuildProject", {
  cache: codebuild.Cache.bucket(cacheBucket, {
    prefix: "maven/", // S3 key prefix for Maven cache
  }),
  // In buildspec: declare what to cache
  // cache:
  // paths:
  // - "/root/.m2/**/*" -- Maven local repository
  // - "**/target/dependency/**/*" -- resolved dependencies
});
// Cache restored before build phase; saved after build completes
// S3 cache object: s3://shopflow-codebuild-cache/maven/cache.zip
// Cache invalidation: change cache key (version bump) or delete S3 object
```

CodePipeline, CodeBuild, and CodeDeploy each assume separate IAM roles with least-privilege permissions. ShopFlow follows separation of concerns: the pipeline role cannot deploy; the deploy role cannot push to ECR; the build role cannot assume the deploy role. Each role has only the permissions needed for its specific stage.

Role	Principal	Key Permissions	ShopFlow Scope
CodePipeline Role	codepipeline.amazonaws.com	s3:GetObject, s3:PutObject (artifacts), codebuild:StartBuild, codebuild:BatchGetBuilds	Pipeline orchestration only
CodeBuild Role (build)	codebuild.amazonaws.com	ecr:GetAuthorizationToken, ecr:BatchCheckLayerAvailability, ecr:BatchGetImage, ecr:PutImage	Build only; Docker push stage, push to ECR
CodeDeploy Role	codedeploy.amazonaws.com	ecs:DescribeServices, ecs:UpdateService, elasticloadbalancing:DescribeLoadBalancers, elasticloadbalancing:ModifyLoadBalancerAttributes	EC2 service; update only
ECS Task Execution Role	ecs-tasks.amazonaws.com	ecr:GetAuthorizationToken, ecr:BatchGetImage (pull from ECR), secretsmanager:GetSecretValue	Tasks ECR, only

CodeBuild Role CDK — Least Privilege

```
const buildRole = new iam.Role(this, "BuildRole", {
  assumedBy: new iam.ServicePrincipal("codebuild.amazonaws.com"),
});
// Only allow ECR push to the specific product-svc repository:
buildRole.addToPolicy(new iam.PolicyStatement({
  effect: iam.Effect.ALLOW,
  actions: ["ecr:GetAuthorizationToken"],
  resources: ["*"], // GetAuthorizationToken requires * resource
}));
buildRole.addToPolicy(new iam.PolicyStatement({
  effect: iam.Effect.ALLOW,
  actions: ["ecr:BatchCheckLayerAvailability", "ecr:PutImage",
```



```
"ecr:InitiateLayerUpload","ecr:UploadLayerPart",
"ecr:CompleteLayerUpload","ecr:BatchGetImage"],
resources: [productRepo.repositoryArn], // ONLY product-svc repo
}});
```

16-13

CodePipeline Parallel Actions for Speed

CodePipeline actions within a stage execute in parallel by default (no runOrder). ShopFlow runs unit tests and static analysis in parallel to reduce pipeline duration. runOrder: 1 runs first; runOrder: 2 runs after all runOrder: 1 actions complete. Parallel build: 8 min → 5 min with unit tests + SAST running simultaneously.

Parallel Stage Actions CDK

```
pipeline.addStage({
  stageName: "Test",
  actions: [
    // runOrder: 1 – all run in parallel
    new codepipeline_actions.CodeBuildAction({
      actionName: "Unit_Tests",
      project: unitTestProject,
      input: sourceOutput,
      runOrder: 1, // parallel with SAST
    }),
    new codepipeline_actions.CodeBuildAction({
      actionName: "SAST_Scan", // SonarQube or Semgrep
      project: sastProject,
      input: sourceOutput,
      runOrder: 1, // parallel with Unit_Tests
    }),
    // runOrder: 2 – runs after all runOrder: 1 complete
    new codepipeline_actions.CodeBuildAction({
      actionName: "Integration_Tests",
      project: integrationTestProject,
      input: sourceOutput,
      runOrder: 2,
    }),
  ],
});
```

Action	runOrder	Duration	Parallel Partner
Unit Tests (Maven Surefire)	1	3 min	SAST Scan (runs simultaneously)
SAST Scan (Semgrep)	1	4 min (controls stage duration)	Unit Tests
Integration Tests (TestContainers)	2	5 min (runs after both runOrder:1 complete)	None (sequential after SAST)

16-14

Release Versioning and Image Tagging Strategy

ShopFlow image tagging: every image tagged with Git commit SHA (unique, immutable) and semantic version tag (v1.2.3) for release builds. :latest tag updated on every main branch push. Production deploys always use Git SHA — never :latest. CDK passes image SHA from build stage to deploy stage via CodePipeline artifact (imageDefinitions.json).

Image Tagging and imageDefinitions.json

```
# In buildspec post_build: write imageDefinitions.json
post_build:
  commands:
    - IMAGE_URI="$AWS_ACCOUNT_ID.dkr.ecr.us-east-1.amazonaws.com/shopflow/product-svc:$CODEBUILD_RESOLVED_SOURCE_VERSION"
    - echo "[{"name":"product-svc","imageUri":"$IMAGE_URI"}]" > imagedefinitions.json
```

```

artifacts:
files:
- imagedefinitions.json
- appspec.yaml # CodeDeploy ECS app spec
- taskdef.json # ECS task definition with new image SHA
# appspec.yaml – CodeDeploy ECS deployment spec:
version: 0.0
Resources:
- TargetService:
Type: AWS::ECS::Service
Properties:
TaskDefinition:
LoadBalancerInfo:
ContainerName: product-svc
ContainerPort: 8080

```

■ `CODEBUILD_RESOLVED_SOURCE_VERSION` is the Git commit SHA of the triggering commit — always the full 40-character SHA. Use it as the image tag for production images. This provides: (1) traceability — any production image can be traced back to the exact commit, (2) immutability — you can redeploy any previous version by referencing its SHA, (3) auditing — CloudTrail shows which SHA was approved and deployed. Never use short SHAs (risk of collision) or `:latest` (not immutable).

16-15

Feature Flags with AppConfig

AWS AppConfig enables runtime feature flag changes without redeployment. ShopFlow uses AppConfig for: enabling/disabling new recommendation engine, A/B testing new checkout flow, and controlling Black Friday features (early access, flash sale timer). AppConfig validates flag schema and supports gradual rollout with automatic rollback on CloudWatch alarm.

AppConfig CDK + Spring Boot Integration

```

// CDK: AppConfig application + environment + configuration profile
const appConfig = new appconfig.CfnApplication(this, "ShopFlowAppConfig", {
name: "shopflow",
});
const prodEnv = new appconfig.CfnEnvironment(this, "ProdEnv", {
applicationId: appConfig.ref,
name: "production",
monitors: [{alarmArn: featureFlagAlarm.alarmArn}],
});
// Feature flag configuration (JSON):
// {
//   "enableRecommendations": false,
//   "checkoutVersion": "v1",
//   "blackFridayBannerEnabled": true,
//   "flashSaleStartTime": "2024-11-29T00:00:00Z"
// }

```

AppConfig Java Client

```

@Component
public class FeatureFlagService {
private final AppConfigDataClient appConfigClient;
private String nextPollToken;
private FeatureFlags cachedFlags;
@Scheduled(fixedDelay = 30_000) // poll every 30 seconds
public void refreshFlags() {
GetLatestConfigurationResponse resp = appConfigClient.getLatestConfiguration(
r -> r.configurationToken(nextPollToken));
nextPollToken = resp.nextPollConfigurationToken();
if (!resp.configuration().asUtf8String().isEmpty()) {
cachedFlags = json.readValue(resp.configuration().asUtf8String(), FeatureFlags.class);
}
}
}

```

```

}
}
public boolean isRecommendationsEnabled() { return cachedFlags.enableRecommendations(); }
}

```

16-16

Pipeline Metrics and DORA Metrics

DORA (DevOps Research and Assessment) metrics measure CI/CD effectiveness: Deployment Frequency, Lead Time for Changes, Change Failure Rate, and Mean Time to Restore. ShopFlow tracks these via CloudWatch custom metrics emitted by a Lambda that queries CodePipeline execution history daily.

DORA Metric	Definition	ShopFlow Target	How Measured
Deployment Frequency	How often code deploys to production	Multiple times per day (Elite)	CodePipeline: count successful prod deploys per day
Lead Time for Changes	Commit to production deploy time	<1 hour (Elite)	CodePipeline: source action start to prod deploy
Change Failure Rate	% of deploys causing prod incident	<5% (Elite)	Failed deploys + rollbacks / total deploys
Mean Time to Restore	Time from incident to recovery	<1 hour (Elite)	PagerDuty: incident open to closed time

DORA Metrics Lambda (daily CloudWatch publish)

```

// Lambda: query last 24h of CodePipeline executions
const executions = await codepipeline.listPipelineExecutions({
  pipelineName: "shopflow-product-svc",
  maxResults: 100,
}).promise();
const deployCount = executions.pipelineExecutionSummaries
  .filter(e => e.status === "Succeeded"
    && e.startTime > yesterday).length;
const failCount = executions.pipelineExecutionSummaries
  .filter(e => e.status === "Failed"
    && e.startTime > yesterday).length;
// Publish to CloudWatch:
await cloudwatch.putMetricData({
  Namespace: "ShopFlow/DORA",
  MetricData: [
    {MetricName:"DeploymentFrequency", Value:deployCount, Unit:"Count"},
    {MetricName:"ChangeFailureRate", Value:failCount/deployCount*100, Unit:"Percent"},
  ],
}).promise();

```

16-17

Canary Deployments with Lambda Aliases

For Lambda functions (SearchIndexer, OrderProcessor), ShopFlow uses Lambda traffic shifting via aliases. Lambda alias can split traffic between two versions: PROD alias: 90% → v5, 10% → v6. CodeDeploy deploys Lambda with CANARY_10PERCENT_10_MINUTES config. If CloudWatch alarms fire during 10 minutes, auto-rollback to 100% v5.

Lambda Canary Deployment CDK

```

// Lambda CodeDeploy application for function-level canary
const lambdaApp = new codedeploy.LambdaApplication(this,"LambdaApp",{
  applicationName: "shopflow-search-indexer",
});
const lambdaDG = new codedeploy.LambdaDeploymentGroup(this,"IndexerDG",{
  application: lambdaApp,
  alias: searchIndexerAlias, // PROD alias
  deploymentConfig: codedeploy.LambdaDeploymentConfig.CANARY_10PERCENT_10MINUTES,
  preHook: preTrafficHookFn, // Lambda: run integration tests on new version
  postHook: postTrafficHookFn, // Lambda: verify metrics after traffic shift
  alarms: [indexerErrorAlarm], // auto-rollback if errors increase
});

```

```
});
```

Hook	When	Purpose	Failure Action
Pre-traffic hook Lambda	Before any traffic shifted to new version	Resiliency integration test against new version directly (via function invocation)	If function fails, do not deploy; no traffic
Post-traffic hook Lambda	After full traffic shifted to new version	Verify error rate and latency in CloudWatch are within bounds	If hooks fails: rollback (all traffic back to old version)

16-18

Pipeline as Code — buildspec Best Practices

buildspec.yaml is the CodeBuild build specification. ShopFlow uses separate buildspec files per stage (buildspec-build.yaml, buildspec-docker.yaml, buildspec-smoke.yaml) committed to the application repo alongside application code. This ensures the build process is versioned and code-reviewed with the application change.

buildspec Best Practices

```
# buildspec.yaml structure:
version: "0.2"

# 1. Use env.parameter-store for config (not hardcoded values):
env:
  parameter-store:
    SONAR_URL: /shopflow/cicd/sonar-url
    secrets-manager:
      SONAR_TOKEN: shopflow/cicd/sonar-token

# 2. Use --batch-mode for Maven (no interactive output):
build:
  commands:
    - mvn verify --batch-mode --no-transfer-progress

# 3. Export test reports to CodeBuild reports (visible in console):
reports:
  product-svc-junit:
    files: ["product-svc/target/surefire-reports/**/*.xml"]
    file-format: JUNITXML
  product-svc-coverage:
    files: ["product-svc/target/jacoco/jacoco.xml"]
    file-format: JACOCOXML

# 4. Fail fast on coverage gate:
- mvn jacoco:check -pl product-svc # fails if coverage < 80%
```

■ Store buildspec files in the application repo (not the CDK/infra repo). This way, a developer changing the service code can also change how it is built and tested in the same pull request — reviewed and approved together. If buildspec is stored in the pipeline CDK repo, there is a synchronization problem: the application code change and the build change are in separate repos and may not be deployed together.

16-19

Dependency Management and Shared Library Pipeline

ShopFlow has shared Java libraries (shopflow-common: DTOs, exceptions, utilities) used by all microservices. A separate CodePipeline builds the shared library, publishes to AWS CodeArtifact (private Maven repository), and updates the BOM (Bill of Materials) version. Microservice pipelines pull the latest BOM version from CodeArtifact.

Component	Tool	Purpose	Update Trigger
Shared library source	GitHub: shopflow-common	shopflow-common-1.2.3.jar (DTOs, exceptions)	PR merge to main
Library pipeline	CodePipeline: shopflow-common-pipeline	Build test + publish to CodeArtifact	GitHub webhook on main push
Artifact repository	AWS CodeArtifact (Maven)	Hosts shopflow-common artifact; upstream: Maven Central	PR merge to main
BOM (Bill of Materials)	shopflow-bom Maven module	Single pom.xml specifying all ShopFlow library versions	Updated in library pipeline after successful build

CodeArtifact CDK

```
const codeArtifactDomain = new codeartifact.CfnDomain(this, "ShopFlowDomain", {
```

```

domainName: "shopflow",
encryptionKey: domainKmsKey.keyArn,
});
const mavenRepo = new codeartifact.CfnRepository(this, "MavenRepo", {
repositoryName: "shopflow-maven",
domainName: codeArtifactDomain.attrName,
externalConnections: ["public:maven-central"], // proxy Maven Central
description: "Private Maven repository for ShopFlow libraries",
});
// In build: authenticate CodeBuild → CodeArtifact:
// CODEARTIFACT_AUTH_TOKEN=$(aws codeartifact get-authorization-token
// --domain shopflow --query authorizationToken --output text)
// mvn --settings settings.xml deploy # settings.xml uses $CODEARTIFACT_AUTH_TOKEN

```

16-20

Black Friday Pipeline Freeze and Deployment Controls

ShopFlow implements a deployment freeze for Black Friday (November 28-30). No new deployments to production during the freeze period. A Lambda function is triggered by EventBridge Scheduler that disables the production stage in CodePipeline. During the freeze, hotfix-only deployments are allowed via a dedicated hotfix pipeline with tighter approval requirements.

Period	Deployment Policy	Approval Required	Who Can Deploy
Normal (all year)	Full pipeline; feature + bug fix deploys	1 approver from release team	Any engineer (PR merged)
Code freeze (Nov 25-27)	Bug fixes only; no new features	2 approvers from release team	Senior engineer + EM approval
Black Friday freeze (Nov 28-30)	Hotfixes only (P1 incidents)	EM + VP Engineering approval	On-call engineer + EM
Post-Black-Friday (Dec 1+)	Normal pipeline resumes	1 approver (normal)	Any engineer

Pipeline Stage Disable Lambda

```

// Lambda: disable production deploy stage (EventBridge Scheduler triggers at Nov 28 00:00 UTC)
await codepipeline.disableStageTransition({
pipelineName: "shopflow-product-svc",
stageName: "Production",
transitionType: "Inbound",
reason: "Black Friday deployment freeze: Nov 28-30. Contact EM for emergency deploys."
}).promise();
// Re-enable (EventBridge Scheduler triggers at Dec 1 06:00 UTC):
await codepipeline.enableStageTransition({
pipelineName: "shopflow-product-svc",
stageName: "Production",
transitionType: "Inbound",
}).promise();

```

■ Disabling a CodePipeline stage transition does not stop the pipeline — it queues execution at the disabled transition boundary. When the transition is re-enabled, the queued execution resumes automatically. This means features merged during the freeze are queued and deploy automatically when the freeze lifts (December 1). Review the queue before re-enabling: `ec2 codepipeline list-pipeline-executions --pipeline-name shopflow-product-svc` to see queued executions.

16-21

SonarQube Code Quality Gate in CodeBuild

SonarQube (hosted on EC2 or SonarCloud SaaS) analyzes code quality metrics: code coverage, duplications, complexity, and security hotspots. ShopFlow integrates SonarQube analysis in the CodeBuild build stage. Pipeline fails if the SonarQube Quality Gate (coverage <80% OR critical security hotspot found) does not pass.

SonarQube Analysis in buildspec

```

# In buildspec-build.yaml build phase:
build:

```

```

commands:
- mvn verify --batch-mode -pl product-svc
- mvn sonar:sonar
-Dsonar.projectKey=shopflow-product-svc
-Dsonar.host.url=$SONAR_URL
-Dsonar.login=$SONAR_TOKEN
-Dsonar.coverage.jacoco.xmlReportPaths=target/jacoco/jacoco.xml
-Dsonar.qualitygate.wait=true # wait for gate result; fail if FAILED

```

SonarQube Metric	ShopFlow Quality Gate Threshold	Current Status	Action if Fails
Line Coverage	>= 80%	83% (passing)	Add unit tests for uncovered code; pipeline blocked
Code Duplications	<= 3%	2.1% (passing)	Extract duplicated code to shared utility
Maintainability Rating	A (no critical smells)	A (passing)	Refactor: code smells flagged in SonarQube UI
Security Hotspots	0 CRITICAL	0 (passing)	Fix security hotspot immediately; P1 incident
Reliability Rating	A (no bugs)	A (passing)	Fix bugs flagged by SonarQube analysis rules

■ `sonar.qualitygate.wait=true` is critical — without it, the Sonar analysis starts but the build completes before the quality gate result is ready (Sonar analysis runs asynchronously on the Sonar server). With `wait=true`, the CodeBuild step polls the Sonar API every 10 seconds until the gate result is available (up to 5 minutes by default) and returns exit code 1 if the gate fails. This blocks the pipeline before the Docker build stage, preventing broken code from being containerized.

16-BP

Best Practices and Anti-Patterns

✓ Pin Docker image tags to immutable SHA (not `:latest`) — `:latest` changes silently between deployments causing undocumented changes

✓ Build once, deploy everywhere — build the Docker image once in the pipeline; pass the image SHA to staging and prod deploys (no rebuilds per environment)

✓ Gate on ECR vulnerability scan — CRITICAL CVEs block deployment automatically; HIGH CVEs alert Slack; treat CVE remediation as P2 incident

✓ Use TestContainers for integration tests — no shared test database; clean state per CI run; real database behavior without mocking

✓ Keep Maven cache in S3 (CodeBuild cache) — reduces build time from 8 min to 4 min by reusing downloaded JARs

✓ Use `CANARY_10PERCENT_5MINUTES` for production deploys — 5-minute window at 10% exposure detects regressions before full rollout

✓ Configure auto-rollback on CloudWatch alarms — tie ALB latency alarm and 5XX alarm to CodeDeploy; rollback within minutes automatically

✓ Log approver identity in CloudTrail — Manual Approval records the IAM ARN of who approved; essential for compliance audit

✓ Run smoke tests against the test listener (port 8443 on green target group) before shifting any production traffic

✓ Use multi-architecture Docker images (`amd64 + arm64`) — single image tag; ECS selects correct arch based on task CPU architecture setting

■ Never commit secrets (AWS keys, DB passwords) to the source repo — use AWS Secrets Manager; scan commits with `git-secrets` or `truffleHog` in CodeBuild

■ Do not use `privileged: false` in CodeBuild for Docker builds — Docker-in-Docker requires privileged mode; without it, docker build fails silently

■ Avoid long-running pipelines (> 20 min) — engineer context-switching cost increases; break into parallel stages where possible

■ Do not skip manual approval for urgent hotfixes — even 5-minute abbreviated approval prevents the most common deployment accidents

Concept	Key Value / Rule
Blue/green vs rolling deploy	Blue/green: 2 target groups; traffic shifts between them; old (blue) tasks run until shift completes; instant r
CodeBuild compute types	SMALL (2 vCPU, 3GB) \$0.005/min, MEDIUM (4 vCPU, 7GB) \$0.01/min, LARGE (8 vCPU, 15GB) \$0.02/m
CodeStar Connection vs GitHub Actions	CodeStar Connection: AWS-managed OAuth to GitHub; supports GitHub Enterprise; branch filtering; trigge
ECR image tag immutability	IMMUTABLE: once pushed, tag cannot be overwritten. Same SHA always references same code. MUTAB
CodeDeploy auto-rollback trigger	Alarm must be in ALARM state during the bake period (5 minutes at 10% traffic for CANARY_10PERCENT
Pipeline artifact bucket	CodePipeline stores artifacts (source zip, imageDefinitions.json, test reports) in S3. Artifact bucket must b